MODUL 223

Projektdokumentation Reservationssystem

Multiuser-Applikationen objektorientiert realisieren

Webanwendung zur Reservierung von Sitzungs- und Veranstaltungsräumen mit Private-/Public-Code, Kollisionsprüfung und SQL-Persistenz.

Projekt	Reservationssystem
Technologie	Spring Boot · Thymeleaf · Spring Data JPA · MySQL
Repository	github.com/FundamentalCoder/Reservationssystem
Version	Dokumentation 4.0
Stand	21.10.2026
Autor	Daniel Santana, Gian Kappeler, Luciano Lüthi

Die Dokumentation beschreibt den aktuellen Stand der gemeinsam realisierten Anwendung und die dazugehörigen UML-/Datenbankmodelle.

Inhalt

1. Projektziel und Ausgangslage 3
2. Anforderungen 3
3. Architektur und Technologien 3
3.1 Projektstruktur 3
4. Datenmodell 4
4.1 Reservation 4
4.2 User 4
5. Controller und Routing 4
6. Geschäftslogik und Validierung 5
6.1 Grundvalidierung 5
6.2 Kollisionsprüfung 5
6.3 Codes 5
7. Benutzeroberfläche mit Thymeleaf 5
8. UML-Klassendiagramm 6
9. UML-Ablaufdiagramm 7
10. Datenbank-UML / ERD 8
11. Fehlerbehandlung und gelöste Probleme 8
12. Testfälle 9
13. Aktueller Stand und mögliche Erweiterungen 9
14. Fazit 9

1. Projektziel und Ausgangslage

In einem Unternehmensgebäude stehen mehrere Räume für Sitzungen und Veranstaltungen zur Verfügung. Das Reservationssystem soll diese Räume verwalten und Doppelbelegungen verhindern. Für die Erstellung einer Reservation ist kein Benutzerkonto notwendig.

Nach dem Speichern erhält der Organisator zwei Codes: Der Private Code dient zum Bearbeiten und Löschen, während der Public Code für die öffentliche Ansicht und die Teilnahme weiterer Personen verwendet wird.

Ziel: Eine einfache, persistente und nachvollziehbare Raumreservation ohne Login, aber mit getrennten Berechtigungen über Private- und Public-Code.

2. Anforderungen

Bereich	Anforderung
Datum	Muss in der Zukunft liegen.
Zeit	Von muss vor Bis liegen.
Zimmer	Nur 101, 102, 103, 104 oder 105.
Bemerkung	10 bis 200 Zeichen.
Teilnehmer	Mindestens Vor- und Nachname; mehrere Personen werden per Komma getrennt.
Belegung	Gleiches Zimmer darf sich am gleichen Datum zeitlich nicht überschneiden.
Codes	Private- und Public-Code werden nach dem Speichern erzeugt.
Persistenz	Reservationen und Teilnehmer werden dauerhaft in der SQL-Datenbank gespeichert.

3. Architektur und Technologien

Die Anwendung folgt einer einfachen Spring-MVC-Struktur. Die Controller übernehmen die Navigation und einen Teil der Geschäftslogik. Die Datenzugriffe erfolgen über ein Spring-Data-JPA-Repository.

Schicht	Komponenten	Aufgabe
View	Thymeleaf-Templates	Formulare, Anzeige von Reservationen und Fehlermeldungen.
Controller	StartController, ReservationController, PrivateController, PublicController	HTTP-Anfragen verarbeiten, Daten prüfen, Seiten auswählen.
Model	Reservation, User	Domänenobjekte und Beziehungen.
Repository	ReservationRepository	Laden und Speichern über JPA/Hibernate.
Datenbank	MySQL	Permanente Speicherung der Reservationen und Teilnehmer.

3.1 Projektstruktur

Controller/ → StartController, ReservationController, PrivateController, PublicController

Model/ → Reservation, User

Repository/ → ReservationRepository

resources/templates/ → index.html, reservation-formular.html, bestaetigung.html, private-reservation.html, reservation-bearbeiten.html, public-reservation.html

4. Datenmodell

4.1 Reservation

Attribut	Typ	Bedeutung
id	Long	Primärschlüssel
datum	LocalDate	Tag der Reservation
von	LocalTime	Startzeit
bis	LocalTime	Endzeit
zimmer	Integer	Raum 101 bis 105
bemerkung	String	Beschreibung / Anlass
privateCode	String	Code für Verwaltung
publicCode	String	Code für öffentliche Ansicht
users	List<User>	Teilnehmer der Reservation

4.2 User

Attribut	Typ	Bedeutung
id	Long	Primärschlüssel
vorname	String	Vorname des Teilnehmers
nachname	String	Nachname des Teilnehmers
reservation	Reservation	Zugehörige Reservation

Zwischen Reservation und User besteht eine 1:n-Beziehung: Eine Reservation kann mehrere Teilnehmer besitzen, ein Teilnehmer gehört genau zu einer Reservation. Die Beziehung ist mit @OneToMany bzw. @ManyToOne umgesetzt.

5. Controller und Routing

Controller	Route	Methode	Aufgabe
StartController	/	GET	Startseite anzeigen
StartController	/code	POST	Private-/Public-Code prüfen und weiterleiten
ReservationController	/reservation/neu	GET	Formular für neue Reservation anzeigen
ReservationController	/reservation	POST	Reservation validieren, Kollisionsprüfung durchführen und speichern
PrivateController	/private	GET	Private Ansicht laden
PrivateController	/private/bearbeiten	GET	Bearbeitungsformular laden
PrivateController	/private/bearbeiten	POST	Änderungen validieren und speichern
PrivateController	/private/loeschen	POST	Reservation löschen
PublicController	/public	GET	Öffentliche Ansicht laden
PublicController	/public/teilnehmen	POST	Teilnehmer zur Reservation hinzufügen

6. Geschäftslogik und Validierung

6.1 Grundvalidierung

Die Klasse Reservation prüft mit isValid(), ob Datum, Start-/Endzeit, Zimmer und Bemerkung gültig sind. Das Datum muss nach dem aktuellen Datum liegen und die Startzeit muss vor der Endzeit liegen.

6.2 Kollisionsprüfung

Vor dem Speichern werden alle Reservationen für dasselbe Datum und Zimmer geladen. Eine Überschneidung liegt vor, wenn die neue Startzeit vor dem Ende einer bestehenden Reservation liegt und die neue Endzeit nach deren Startzeit liegt.

```
Überschneidung = neueVon < bestehendBis  UND  neueBis > bestehendVon
```

Beim Bearbeiten wird dieselbe Prüfung durchgeführt. Die aktuell bearbeitete Reservation wird anhand ihrer ID übersprungen, damit sie nicht mit sich selbst kollidiert.

6.3 Codes

Private- und Public-Code werden beim Erstellen mit UUID.randomUUID().toString() erzeugt. Dadurch erhält jede Reservation zwei unterschiedliche Zugriffsmöglichkeiten.

7. Benutzeroberfläche mit Thymeleaf

Template	Funktion
index.html	Startseite, neue Reservation starten, Code eingeben
reservation-formular.html	Datum, Zeit, Zimmer, Bemerkung und Teilnehmer erfassen
bestaetigung.html	Gespeicherte Daten sowie Private-/Public-Code anzeigen
private-reservation.html	Reservation mit Verwaltungsfunktionen anzeigen
reservation-bearbeiten.html	Datum, Zeit, Zimmer und Bemerkung bearbeiten
public-reservation.html	Reservation ansehen und als Teilnehmer beitreten

8. UML-Klassendiagramm

Das Klassendiagramm zeigt die vier Controller, die beiden Model-Klassen und das Repository. Die gestrichelten Pfeile stellen Abhängigkeiten dar. Zwischen Reservation und User besteht eine 1:n-Beziehung.

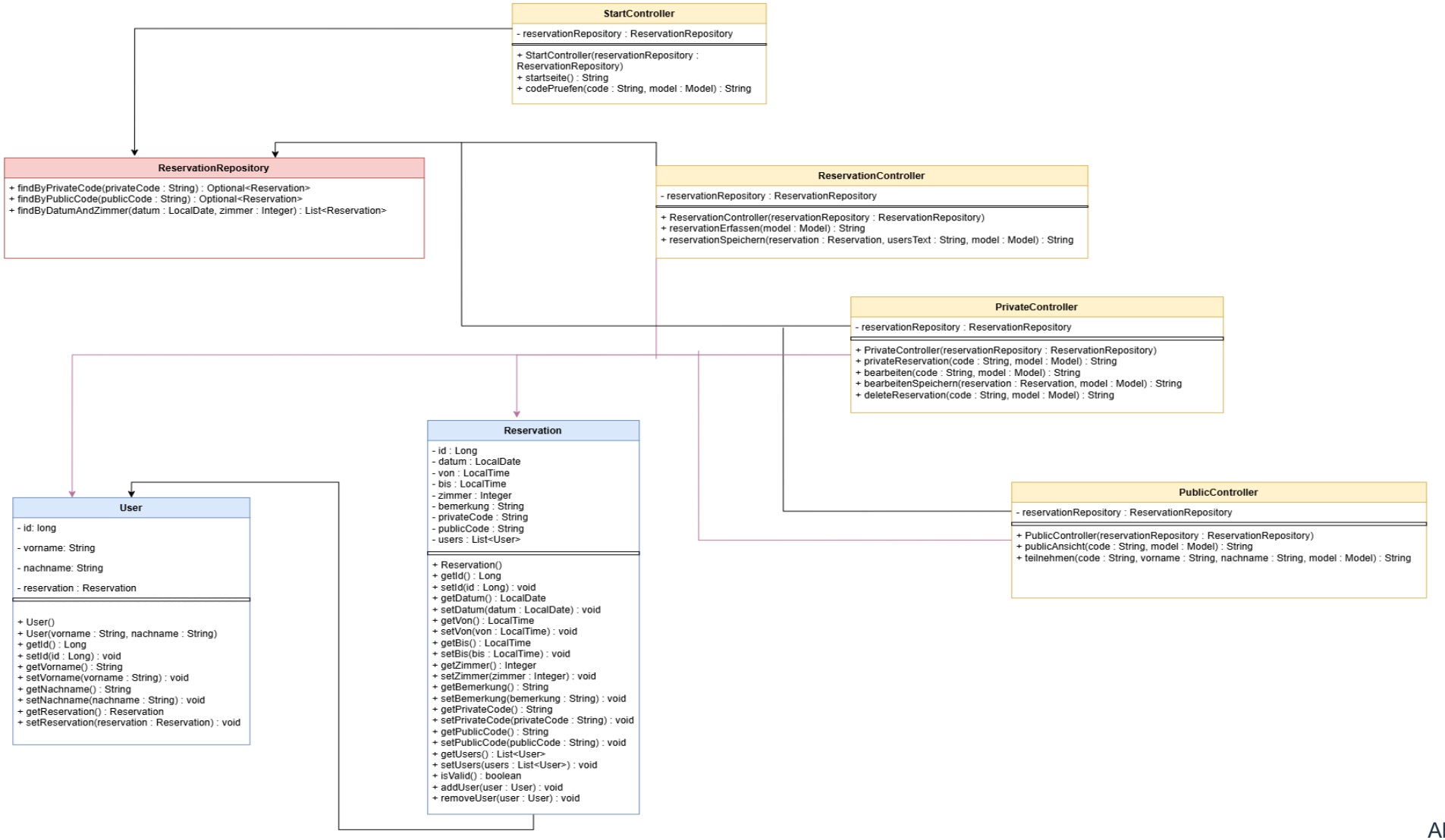


Abbildung 1: UML-Klassendiagramm des Reservationssystems

9. UML-Ablaufdiagramm

Das Zustandsdiagramm beschreibt die Navigation zwischen den Seiten. Es zeigt die Entscheidungen für gültige oder ungültige Codes, die Reservationserstellung sowie die getrennten Private- und Public-Abläufe.

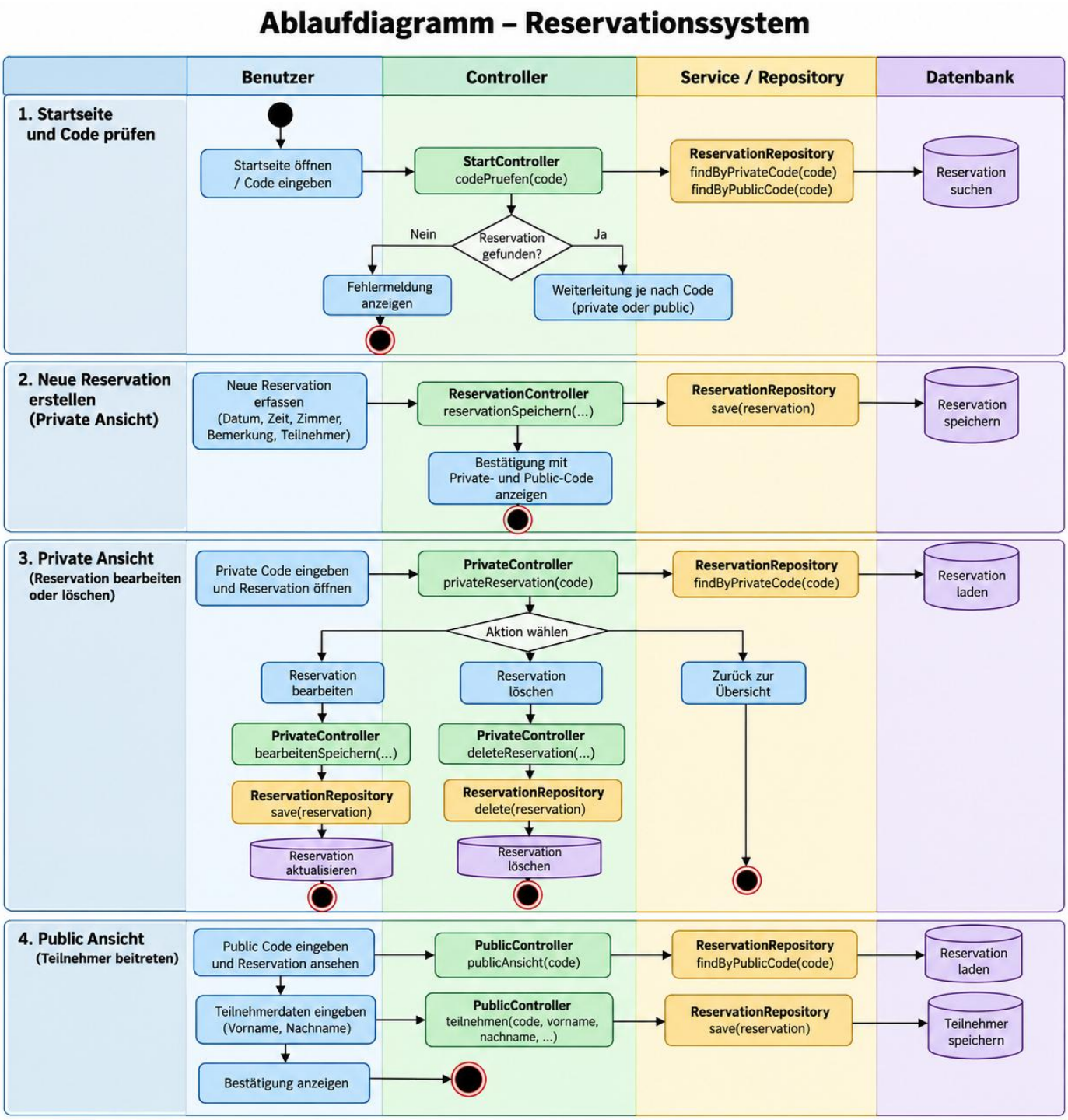


Abbildung 2: UML-Zustandsdiagramm der Webnavigation

10. Datenbank-UML / ERD

Das Datenbankmodell besteht aus zwei Tabellen. Die Tabelle reservationen enthält die Reservierungsdaten und die beiden Codes. Die Tabelle user enthält die Teilnehmer und verweist über reservation_id auf die zugehörige Reservation.

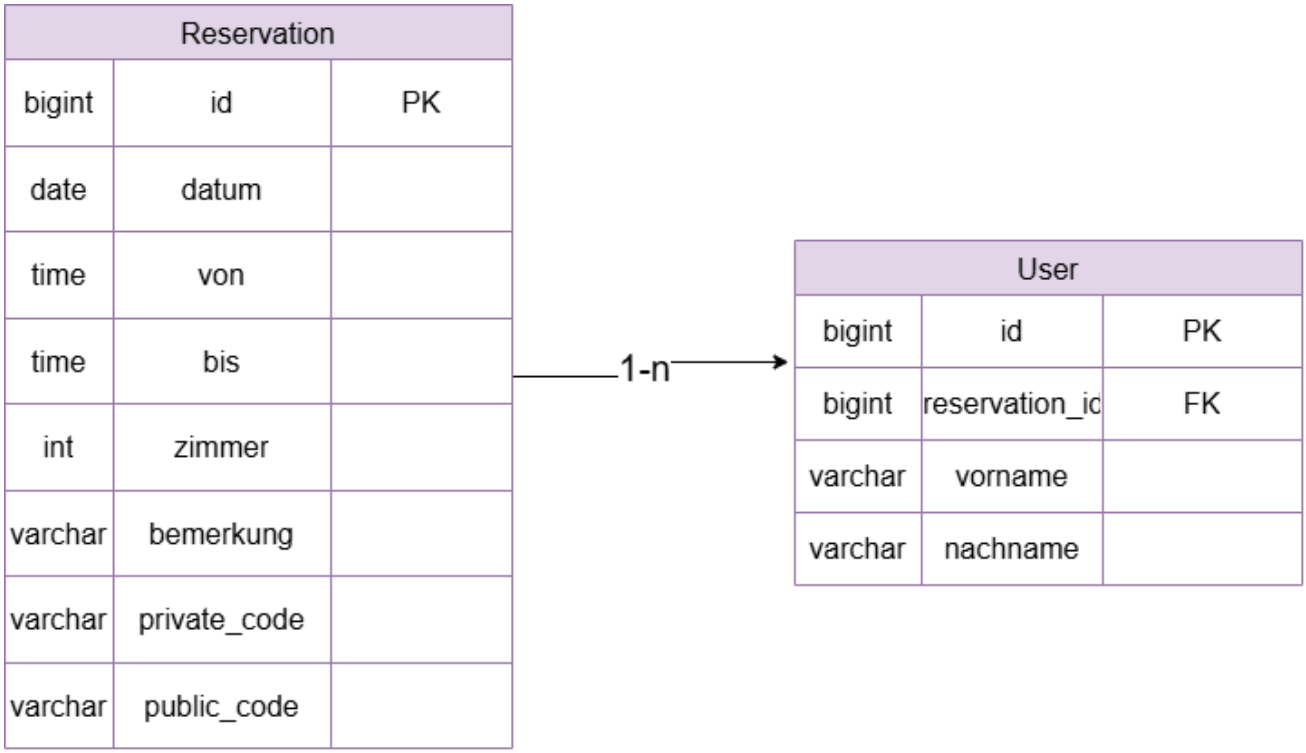


Abbildung 3: Datenbank-UML / ERD mit Primär- und Fremdschlüssel

Kardinalität: reservationen 1 : n user. Das Fremdschlüsselfeld user.reservation_id stellt die Beziehung her.

11. Fehlerbehandlung und gelöste Probleme

Problem	Ursache	Lösung
Thymeleaf: reservation.users nicht gefunden	Model hatte zeitweise getUser()/setUser() statt getUsers()/setUsers().	Attribut und Getter/Setter auf users vereinheitlicht.
404 bei Bearbeiten	Link zeigte auf /reservation/bearbeiten statt /private/bearbeiten.	Route im HTML an den PrivateController angepasst.
405 Method Not Allowed	Für /private/bearbeiten existierte zunächst nur POST.	Zusätzliche GET-Methode zum Anzeigen des Formulars erstellt.
Template reservation-bearbeiten fehlt	Controller gab einen nicht vorhandenen Template-Namen zurück.	reservation-bearbeiten.html im templates-Ordner angelegt.
Doppelte Raumbelegung möglich	Kollisionsprüfung fehlte in reservationSpeichern().	findByDatumAndZimmer() und Zeitüberschneidungsprüfung ergänzt.
Kollision beim Bearbeiten	Bearbeitung muss ebenfalls auf Überschneidungen prüfen.	Gleiche Prüfung ergänzt und eigene Reservation per ID ausgeschlossen.

12. Testfälle

Nr.	Test	Erwartetes Ergebnis
1	Neue gültige Reservation erstellen	Reservation wird gespeichert; Private-/Public-Code werden angezeigt.
2	Datum heute oder Vergangenheit	Eingabe wird abgelehnt.
3	Von-Zeit nach Bis-Zeit	Eingabe wird abgelehnt.
4	Zimmer 106 auswählen / manipulieren	Validierung lehnt Reservation ab.
5	Gleiches Zimmer, überlappende Zeit	Reservation wird nicht gespeichert; Fehlermeldung erscheint.
6	Gleiches Zimmer, direkt anschliessende Zeit	Reservation ist erlaubt.
7	Gültigen Private Code eingeben	Private Ansicht wird geöffnet.
8	Reservation über Private Code bearbeiten	Änderungen werden gespeichert, wenn sie gültig und kollisionsfrei sind.
9	Reservation über Private Code löschen	Reservation wird gelöscht.
10	Gültigen Public Code eingeben	Öffentliche Ansicht wird geöffnet.
11	Teilnehmer über Public Ansicht hinzufügen	Teilnehmer wird gespeichert und angezeigt.
12	Ungültigen Code eingeben	Fehlermeldung auf der Startseite.

13. Aktueller Stand und mögliche Erweiterungen

- Erstellen, Anzeigen, Bearbeiten und Löschen von Reservationen ist umgesetzt.
- Private-/Public-Code funktionieren als getrennte Zugriffsmöglichkeiten.
- Kollisionsprüfung für neue und bearbeitete Reservationen ist umgesetzt.
- Teilnehmer werden mit der Reservation gespeichert; Public-Benutzer können sich selbst hinzufügen.
- Mögliche Erweiterung: Teilnehmer auch im Private-Bearbeitungsformular vollständig verwalten.
- Mögliche Erweiterung: Eindeutige Datenbank-Constraints für privateCode und publicCode.
- Mögliche Erweiterung: Validierung der Teilnehmernamen und Schutz vor doppelten Einträgen.
- Mögliche Erweiterung: Service-Schicht zur Trennung der Geschäftslogik von den Controllern.

14. Fazit

Das Reservationssystem erfüllt die zentralen Anforderungen des Projekts: Räume können ohne Benutzerkonto reserviert werden, die Daten werden dauerhaft gespeichert, Zeitüberschneidungen werden verhindert und die Verwaltung erfolgt über getrennte Private- und Public-Codes. Die Architektur bleibt für ein Schulprojekt bewusst übersichtlich und basiert auf Spring MVC, JPA/Hibernate und Thymeleaf.

Während der Umsetzung wurden typische Probleme bei Routing, HTTP-Methoden, Thymeleaf-Binding und JPA-Beziehungen gelöst. Die daraus entstandene Struktur ist nachvollziehbar und kann später um zusätzliche Validierungen, eine Service-Schicht oder weitere Funktionen erweitert werden.